

## EXPERIMENT 6

**Student Name: SUMIT CHAUHAN**

**Branch: AIT-CSE (AIML)**

**Semester: 4**

**Subject Name: Database Management System**

**UID: 24BAI70884**

**Section/Group: 24AIT\_KRG-G2**

**Subject Code: 24CSH-298**

### **Experiment 6 – Understanding and Implementing Cursors for Row-by-Row Data Processing (Oracle, Accenture, Scaler, D. E. Shaw & Co)**

#### **Experiment**

Experiment 6: Working with PL/SQL cursors to process multiple rows of a table individually. This experiment demonstrates the use of implicit cursors, explicit cursors, and cursor attributes to implement business logic on row-by-row data in Oracle and PostgreSQL.

---

#### **Aim**

To understand the concept and working of cursors in PL/SQL for row-by-row data processing, and to analyze how implicit cursors, explicit cursors, and cursor attributes are used to implement business logic on multiple rows in a database table.

---

#### **Objective**

- To implement and analyze implicit cursors for single-row operations.
  - To implement explicit cursors to fetch and process multiple records.
  - To apply cursor attributes (%FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN) for controlling program flow.
  - To process employee records row by row and apply business logic effectively.
- 

#### **Software Requirements**

##### **Database Management System:**

- Oracle Database Express Edition (Oracle XE)

## EXPERIMENT 6

- PostgreSQL Database

### Database Administration Tool / Client Tool:

- Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)
- 

### Problem Statement

In real-world enterprise applications, database queries often return multiple rows that need to be processed individually to apply specific business rules. Using cursors allows row-by-row processing for detailed business logic implementation.

---

### Practical / Experiment Steps

1. Design PL/SQL programs that demonstrate:
    - The use of implicit cursors for single-row DML operations
    - The use of explicit cursors to fetch and process multiple records
    - The application of cursor attributes to control program execution
  2. Create PL/SQL programs to:
    - Fetch employee records from a database table using cursors
    - Process each record individually
    - Display results or apply business logic using cursor attributes
- 

### Procedure

1. Open Oracle SQL Developer or pgAdmin and connect to the database.
2. Create an employee table with sample data (id, name, department, salary).
3. Write an anonymous PL/SQL block demonstrating implicit cursor operations.
4. Write a PL/SQL block using an explicit cursor to fetch and process multiple records.
5. Apply cursor attributes such as %FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN to control execution flow.

## EXPERIMENT 6

6. Execute the blocks and observe the output.
7. Capture results and screenshots for documentation.

---

### Input / Output Details

#### Input

##### employee table:

- id (Integer)
- name (Varchar)
- department (Varchar)
- salary (Integer)

##### Cursor logic used:

- Implicit cursor for single-row processing
- Explicit cursor for multi-row processing
- Cursor attributes (%FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN)

---

### Step-wise Output

## EXPERIMENT 6

### Step 1 – Create employee Table

[ SQL Worksheet ]\*      Aa 

```
1 CREATE TABLE EMPLOYEE (  
2     EMP_ID NUMBER PRIMARY KEY,  
3     EMP_NAME VARCHAR2(50),  
4     SALARY NUMBER  
5 );  
6 |
```

Query result **Script output** DBMS output Explain Plan SQL history

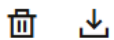


Table EMPLOYEE dropped.

Elapsed: 00:00:00.035

```
SQL> CREATE TABLE EMPLOYEE (  
      EMP_ID NUMBER PRIMARY KEY,  
      EMP_NAME VARCHAR2(50),  
      SALARY NUMBER...
```

[Show more...](#)

Table EMPLOYEE created.

Elapsed: 00:00:00.012

## EXPERIMENT 6

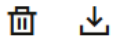
### Step 2 – Insert Data into employee Table

[ SQL Worksheet ]\*      Aa 



```
1 CREATE TABLE EMPLOYEE (  
2     EMP_ID NUMBER PRIMARY KEY,  
3     EMP_NAME VARCHAR2(50),  
4     SALARY NUMBER  
5 );  
6 INSERT INTO EMPLOYEE VALUES (101, 'Amit', 25000);  
7 INSERT INTO EMPLOYEE VALUES (102, 'Riya', 32000);  
8 INSERT INTO EMPLOYEE VALUES (103, 'Rahul', 45000);  
9 INSERT INTO EMPLOYEE VALUES (104, 'Sneha', 28000);  
10 INSERT INTO EMPLOYEE VALUES (105, 'Arjun', 50000);  
11  
12 COMMIT;
```

Query result Script output DBMS output Explain Plan SQL history



1 row inserted.

Elapsed: 00:00:00.002

SQL> COMMIT



Commit complete.

Elapsed: 00:00:00.002

## EXPERIMENT 6

### Step 3 – Implicit Cursor Operation

[ SQL Worksheet ]\*

```

13
14 BEGIN
15     ... UPDATE EMPLOYEE
16     ... SET SALARY = SALARY * 1.10
17     ... WHERE SALARY < 30000;
18
19     DBMS_OUTPUT.PUT_LINE('Rows Updated: ' || SQL%ROWCOUNT);
20
21     IF SQL%FOUND THEN
22         ... DBMS_OUTPUT.PUT_LINE('Update Successful');
23     END IF;
24 END;
25 /

```

Query result
Script output
DBMS output
Explain Plan
SQL history

SQL> BEGIN  
UPDATE EMPLOYEE  
SET SALARY = SALARY \* 1.10  
WHERE SALARY < 30000;...  
Show more...

Rows Updated: 2  
Update Successful

PL/SQL procedure successfully completed.  
Elapsed: 00:00:00.091

### Step 4 – Explicit Cursor Declaration and Fetch

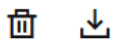
## EXPERIMENT 6

```

26
27 DECLARE
28     CURSOR emp_cursor IS
29         SELECT EMP_ID, EMP_NAME, SALARY FROM EMPLOYEE
30         WHERE SALARY > 30000;
31
32     v_id EMPLOYEE.EMP_ID%TYPE;
33     v_name EMPLOYEE.EMP_NAME%TYPE;
34     v_salary EMPLOYEE.SALARY%TYPE;
35
36 BEGIN
37     OPEN emp_cursor;
38
39     LOOP
40         FETCH emp_cursor INTO v_id, v_name, v_salary;
41         EXIT WHEN emp_cursor%NOTFOUND;
42

```

Query result   **Script output**   DBMS output   Explain Plan   SQL history



```

SQL> DECLARE
      CURSOR emp_cursor IS
          SELECT EMP_ID, EMP_NAME, SALARY FROM EMPLOYEE
          WHERE SALARY > 30000;...
Show more...

```

```

ID: 102 Name: Riya Salary: 32000
ID: 103 Name: Rahul Salary: 45000
ID: 104 Name: Sneha Salary: 30800
ID: 105 Name: Arjun Salary: 50000

```

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.079

## EXPERIMENT 6

[ SQL Worksheet ]\*

```

54 DECLARE
55     CURSOR emp_cursor IS SELECT * FROM EMPLOYEE;
56     emp_record EMPLOYEE%ROWTYPE;
57
58 BEGIN
59     OPEN emp_cursor;
60
61     IF emp_cursor%ISOPEN THEN
62         DBMS_OUTPUT.PUT_LINE('Cursor Opened Successfully');
63     END IF;
64
65     LOOP
66         FETCH emp_cursor INTO emp_record;
67         EXIT WHEN emp_cursor%NOTFOUND;
68
69         DBMS_OUTPUT.PUT_LINE(
70             'Employee: ' || emp_record.EMP_NAME ||

```

Query result
Script output
DBMS output
Explain Plan
SQL history

...  
Show more...

Cursor Opened Successfully  
Employee: Amit | Salary: 27500  
Employee: Riya | Salary: 32000  
Employee: Rahul | Salary: 45000  
Employee: Sneha | Salary: 30800  
Employee: Arjun | Salary: 50000  
Total Rows Processed: 5

PL/SQL procedure successfully completed.  
Elapsed: 00:00:00.020

[Terms and Conditions](#) | 
[Your Privacy Rights](#) | 
[Delete Your FreeSQL Account](#) | 
[Cookie Preferences](#)

26.1.2 - Copyright © 20

### Learning Outcome

After completing this experiment, the learner will be able to:

- Understand the role of cursors in PL/SQL for handling multi-row query results.
- Differentiate between implicit cursors and explicit cursors.
- Use cursor attributes such as %FOUND, %NOTFOUND, %ROWCOUNT, and %ISOPEN.
- Develop PL/SQL programs that process database records row by row.





## EXPERIMENT 6

- Apply cursor-based logic in real-world business scenarios used in organizations like Oracle, Accenture, Scaler, and D. E. Shaw & Co.